SOCIETY FOR COMPUTER TECHNOLOGY & RESEARCH'S
PUNE INSTITUTE OF COMPUTER TECHNOLOGY
AN AUTONOMOUS INSTITUTE AFFILIATED TO SPPU, PUNE
DEPARTMENT OF INFORMATION TECHNOLOGY
40-Hours Value Added Hybrid Certificate Course on 'Java Backend Development using Spring Boot'
16th March to 6th April 2026

Instructions for Core Java Problem Statement (Implementation & Submission)

Dear Participants,

Please follow the below instructions carefully for the problem statement:

Read the problem statement carefully before starting. Follow proper OOP concepts (Encapsulation, Inheritance, Polymorphism). Write clean, structured, and well-indented code. Use appropriate class names, methods, and comments where necessary. Ensure your program runs without errors

Submission Guidelines
• Save your Java file with the format: *Name.java*
• Create a *Word/PDF document* named: *YourName_Problem1* and *YourName_Problem2*
• In the document, include:

- Your complete program code
- Screenshots of the output
  • Upload the document on the *provided Google Form* within the deadline
- Google Form Link: https://forms.gle/XNdyi7MrF2rbihbT6
- Problem Statements are given below.
- Submission Date: 30/3/2026

Important Notes
• Late submissions will *not* be accepted (Last Date: 30/3/2026 Time: Till 6pm)
• Incomplete or non-working code may affect evaluation
• Avoid copying—individual work is expected

Course Coordinators,
Mrs. Amruta Patil
Ms. Jyoti Jadhav
Ms. Prajakta Shinde
Mrs. Deepali Salapurkar

Problem Statement 1: Student Management System (Core Java OOP + Collections)

Objective

Design a Java-based **Student Management System** to manage student records using **Encapsulation, Abstraction, Inheritance, Polymorphism, and Collections (Map)**.

Requirements

1. Create an **abstract class** Student with:
    - Attributes:
        - studentId
        - name
        - course
        - baseFee
    - Use **private access modifiers** and provide getters/setters (Encapsulation)
    - Define an **abstract method**:
      double calculateFee();
2. Implement subclasses:
    - RegularStudent
    - ScholarshipStudent
    - ExchangeStudent
3. Fee Calculation Logic:
    - **RegularStudent**: baseFee
    - **ScholarshipStudent**: baseFee - (20% discount)
    - **ExchangeStudent**: baseFee + international charges (fixed amount)
4. Use a Map<Integer, Student> to:
    - Store student records (studentId as key)
    - Perform the following operations:
        - Add a student
        - Search student by ID
        - Update student details
        - Delete student record
5. Demonstrate **Polymorphism** by:
    - Storing different student types in the same Map
    - Calling calculateFee() for each student object

Expected Outcome

- Understanding of:
    - **Encapsulation & Abstraction**
    - **Inheritance & Runtime Polymorphism**
- Practical use of:
    - **Map and Generics in Java**
- Ability to design a modular and scalable system

Problem Statement 2: Smart HR System with Advanced Operations

Objective

Build a **Smart HR System** that manages employees and supports advanced operations using **Collections Framework (List, Set, Map)** and OOP principles.

Requirements

1. Create a base class Employee with:
    - Fields: empId, name, email, role
    - Proper encapsulation
2. Extend into:
    - PermanentEmployee
    - ContractEmployee
    - InternEmployee
3. Add unique behaviors:
    - Permanent: yearly bonus + benefits
    - Contract: hourly pay × hours worked
    - Intern: stipend + performance bonus
4. Use Collections:
    - Map<Integer, Employee> → store all employees
    - List<Employee> → display all employees
    - Set<String> → maintain unique departments or roles
5. Functionalities to implement:
    - Add multiple employees
    - Display all employees
    - Search by name or ID
    - Update employee role/salary
    - Delete employee
    - Display all unique roles (using Set)
6. Apply **Generics** properly for all collections.

Expected Outcome

- Strong grasp of **Inheritance & Polymorphism**
- Practical use of **List, Set, Map**
- Real-world simulation of HR system
- Clean and modular Java design